

Deep Learning Statistical Arbitrage: An Empirical Study of Neural Trading Policies on Equity Residuals

Hadis Surya Al Amin¹

¹Project report – based on the framework of Guijarro-Ordóñez, Pelger, and Zanotti (2021)

May 29, 2026

Abstract

We study data-driven statistical arbitrage in U.S. equities by learning trading policies directly from *residual* return series. Residuals are extracted with three families of factor models—IPCA, PCA, and the Fama–French five-factor model—and a neural trading policy is trained end-to-end to maximize an out-of-sample Sharpe ratio. We evaluate a convolution–transformer architecture with a graph component (**CNNTransformer**) against feed-forward baselines (**RawFFN**, **FourierFFN**, **OUFFN**). This report describes the data, methodology, and empirical results, including risk-adjusted performance and turnover across factor models.

1 Introduction

Classical statistical arbitrage exploits temporary mispricings in the residual component of asset returns—the part left after systematic (factor) exposure is removed. Rather than assuming a parametric mean-reverting process (e.g. an Ornstein–Uhlenbeck model) and hand-tuning entry/exit thresholds, we follow the *Deep Learning Statistical Arbitrage* framework and learn the trading policy directly from data. The policy maps a window of recent residual behavior to a portfolio weight, and is optimized to maximize a risk-adjusted objective on out-of-sample (OOS) data via rolling retraining.

2 Data and Residual Construction

The investable universe is defined by a market-capitalization cap proportion (`cap_proportion = 0.01`). For each asset we form a residual time series by removing common factors using one of three factor models:

- **IPCA** — Instrumented Principal Component Analysis (here 5 factors),
- **PCA** — rolling-window principal component analysis (5 factors),
- **Fama–French** — the daily five-factor model.

Daily trading dates are aligned to the Fama–French research data over the OOS period (1998–2017). When `use_residual_weights` is enabled, the residual composition matrix is used to translate residual-space positions into asset-space weights for realistic turnover and short-proportion accounting.

3 Methodology

3.1 Trading policy models

We compare four trading-policy architectures, all consuming a lookback window of preprocessed residuals (cumulative-sum normalized):

- **RawFFN** — feed-forward network on raw residual windows (baseline).
- **FourierFFN** — feed-forward network on Fourier features of the window.
- **OUFFN** — feed-forward network with an Ornstein–Uhlenbeck-inspired layer.
- **CNNTransformer** — a one-dimensional convolutional encoder followed by a multi-head self-attention (transformer) block, optionally augmented by a fully connected graph neural network (GNN) component to share information across the cross-section.

The reported **CNNTransformer** configuration uses a lookback of 30 days, dropout 0.25, convolutional filters [1, 8] of size 2, 4 attention heads, and a single GNN layer with hidden dimension 8.

3.2 Objective and training protocol

Let r_t denote the daily portfolio return generated by the policy. The policy parameters θ are trained to maximize the in-sample Sharpe ratio

$$\mathcal{L}(\theta) = \frac{\mathbb{E}[r_t]}{\sqrt{\text{Var}[r_t]}}, \quad (1)$$

optionally net of transaction costs (**trans_cost**) and holding/short costs (**hold_cost**). Training uses Adam (learning rate 10^{-3} , batch size 32) for up to 100 epochs. We employ a **rolling-window** protocol: the model is retrained on a window of **length_training** = 1000 trading days and applied to the subsequent block, with retraining every **retrain_freq** = 125 days, so all reported performance is strictly out-of-sample.

3.3 Evaluation metrics

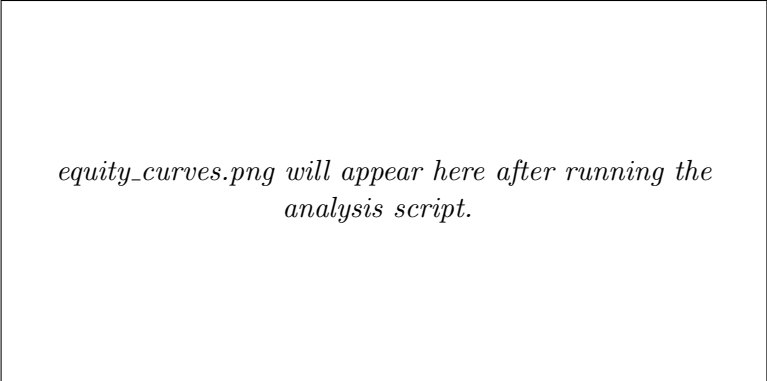
For each (model, factor model) pair we report the annualized Sharpe ratio ($\text{daily Sharpe} \times \sqrt{252}$), mean daily return, return standard deviation, turnover, and short proportion.

4 Results

Table ?? summarizes risk-adjusted performance across factor models, sorted by annualized Sharpe ratio. Figure ?? shows the corresponding out-of-sample equity curves (cumulative return, log scale).

Table 1: Out-of-sample performance by trading policy and factor model.

Run `python3 tools/analyze_results.py` to generate this table.



*equity_curves.png will appear here after running the
analysis script.*

Figure 1: Out-of-sample cumulative returns (equity curves) for each trading policy, log scale, normalized to a starting value of 1.0.

4.1 Discussion

We discuss which factor-model residuals yield the strongest risk-adjusted returns, the trade-off between Sharpe ratio and turnover, and the marginal value of the transformer and GNN components relative to the feed-forward baselines.

5 Conclusion

We presented an end-to-end deep learning approach to statistical arbitrage on equity residuals and benchmarked several neural trading-policy architectures across IPCA, PCA, and Fama–French factor models under a rolling out-of-sample protocol.

Reproducibility

Results are produced with `run_train_test.py` using the configuration files in `configs/`. To regenerate the table and figure in this report:

```
python3 tools/analyze_results.py          # all models
python3 tools/analyze_results.py -m CNNTransformer
```

To match the published paper, set `use_residual_weights: True` in the config.

References

- [1] J. Guijarro-Ordóñez, M. Pelger, and G. Zanotti. *Deep Learning Statistical Arbitrage*, 2021. <https://arxiv.org/abs/2106.04028>.